

77. Shadow DOM Styling

Shadow DOM me `<style>` aur `<link rel="stylesheet" href="...">` dono ho sakte hain. `<link>` wale case me stylesheets HTTP-cached hoti hain, isliye ek hi template use karne wale kai components ke liye dobara download nahi hoti.

General rule: local styles sirf shadow tree ke andar kaam karte hain, aur document styles bahar. Par kuch exceptions hain.

`:host`

Shadow host (wo element jisme shadow tree hai) ko select karta hai. Jaise `<custom-dialog>` ko center karna hai to us element ko khud style karna hoga, wahi `:host` karta hai:

```
html

<template id="tpl">
  <style>
    :host {
      position: fixed;
      left: 50%;
      top: 50%;
      transform: translate(-50%, -50%);
      display: inline-block;
      border: 1px solid red;
      padding: 10px;
    }
  </style>
  <slot></slot>
</template>
```

Cascading

Shadow host (`<custom-dialog>`) light DOM me rehta hai, isliye document CSS rules uspar lagte hain. Agar koi property `:host` (local) aur document dono me style ho, to **document wali style jeetti hai**.

CSS

```
custom-dialog { padding: 0; } /* document me: to dialog bina padding ke */
```

Isse "default" component styles `:host` me rakh ke document me aasani se override kar sakte hain. **Exception:** local property `!important` ho to local jeetti hai.

`:host(selector)`

`:host` jaisa par sirf tab jab shadow host `(selector)` se match kare. Jaise sirf `(centered)` attribute wale dialog ko center karna:

CSS

```
:host([centered]) {  
  position: fixed;  
  left: 50%; top: 50%;  
  transform: translate(-50%, -50%);  
  border-color: blue;  
}  
  
:host {  
  display: inline-block;  
  border: 1px solid red;  
  padding: 10px;  
}
```

html

```
<custom-dialog centered>Centered!</custom-dialog>  
<custom-dialog>Not centered.</custom-dialog>
```

Yaani `:host` family se component ke main element ko style karo. Ye styles `(!important)` ke bina) document se override ho sakti hain.

Slotted content ko style karna

Slotted elements **light DOM** se aate hain, to **document styles** use karte hain. **Local styles** **slotted content** par nahi lagte.

html

```
<style> span { font-weight: bold } </style>  
  
<user-card>  
  <div slot="username"><span>John Smith</span></div>  
</user-card>  

```

1. `<slot>` ko hi style karo aur CSS inheritance par bharosa (par sab properties inherit nahi hoti):

CSS

```
slot[name="username"] { font-weight: bold; }
```

2. `::slotted(selector)` pseudo-element. Ye tab match karta hai jab: (1) wo light DOM ka slotted element ho (slot ka naam matter nahi), aur (2) `selector` se match kare. Sirf **element khud**, uske children nahi:

CSS

```
::slotted(div) { border: 1px solid red; }
```

`::slotted` aur andar nahi ghus sakta. Ye **invalid** hain:

CSS

```
::slotted(div span) { }      /* nahi chalega */
::slotted(div) p { }        /* light DOM ke andar nahi ja sakte */
```

Aur `::slotted` sirf CSS me hota hai, `querySelector` me nahi.

Custom properties se CSS hooks

Main document se component ke **internal elements** ko kaise style karein? `:host` sirf `<custom-dialog>`/`<user-card>` par asar karta hai, andar ke shadow DOM elements par nahi. Document se seedha shadow DOM styles par asar karne wala koi selector nahi hai.

Par jaise hum component ke methods expose karte hain, waise **CSS variables (custom properties)** expose kar sakte hain. **Custom CSS properties har level par hoti hain (light aur shadow dono me), aur shadow DOM ko "pierce" karti hain.**

Shadow DOM me:

CSS

```
.field {
  color: var(--user-card-field-color, black); /* define na ho to black */
}
```

Bahar ke document me:

CSS

```
user-card { --user-card-field-color: green; }
```

Andar ka `.field` rule ise use kar leta hai.

Summary

Local styles asar karte hain:

- shadow tree par
- shadow host par (`:host` aur `:host()`)
- slotted elements par (`::slotted(selector)`), sirf element khud, uske children nahi)

Document styles asar karte hain:

- shadow host par (wo bahar ke document me hai)
- slotted elements aur unke content par (wo bhi bahar ke document me)

Conflict me normally **document styles jeetti hain**, jab tak property `!important` na ho (tab local jeetti hai).

Custom properties shadow DOM ke aar-paar jaati hain, inhe component ke "hooks" ki tarah use karo:

1. Component key elements ko `var(--component-name-title, <default>)` jaise custom property se style karta hai.
2. Component ka author ye properties developers ke liye publish karta hai (ye bhi public methods ki tarah important).
3. Developer title style karna chahe to shadow host ya uske upar `--component-name-title` set kare.
4. Kaam ho gaya!